CS360 Introduction to Database

Algebraic and Logical Query Languages

Min-Soo Kim

School of Computing, KAIST

High-level programming for relational DBs

- Two abstract programming languages
  - algebraic language (called relational algebra, imperative)
  - logic-based language (called Datalog, declarative)
- We extend relational algebra
  - set-based to bags to better reflect the way the relational model is implemented in DBMS's
  - to handle several more operations (e.g., aggregations)
- Datalog
  - allows us to express queries by describing the desired results
  - rather than by giving an algorithm to compute the results

Extending relational algebra

Relational operations on bags

- Bags (multisets)
  - allow the same tuple to appear more than once in a relation
- When relations are bags (not sets)
  - we need to change the definition of some relational operations
- e.g., a temporary relation during query processing
  - one of the main reasons why actual DBMSs use the bag-model
  - the order of tuples does not matter like sets

A	B
1	2
3	4
1	2
1	2

Why bags?

- Commercial DBMS's implement relations that are bags
  - motivation: some relational operations are considerably more efficient if we use the bag model
  - e.g., union of two relations as bags simply needs to copy one relation and add to the copy of the other relation
    - no need to eliminate duplicate copies of a tuple
  - e.g., projection as bags simply projects each tuple and add it to the result
    - c.f., projection as sets needs to compare each projected tuple with all the other projected tuples
  - e.g., AVG (column A) for sets is 2
    - (1+3) / 2 = 2
  - e.g., AVG (column A) for bags is 1.5
    - (1+3+1+1) / 4 = 1.5

A	B	C
1	2	5
3	4	6
1	2	7
1	2	8

$\pi_{A,B}(R)$

Union, intersection, and difference of Bags

- Suppose that R and S are bags
  - tuple t appears n times in R and m times in S
- Bag union $R \cup S$: t appears (n+m) times
- Bag intersection $R \cap S$: t appears min(n,m) times
- Bag difference $R - S$: t appears max(0,n-m) times
  - intuitively, each occurrence of t in S cancels one occurrence in R

R

A	B
1	2
3	4
1	2
1	2

S

A	B
1	2
3	4
3	4
5	6

A	B
1	2
3	4
1	2
1	2
1	2
3	4
3	4
5	6

$R \cup S$

A	B
1	2
3	4

$R \cap S$

A	B
1	2
1	2

$R - S$

n = 3
m = 1

A	B
3	4
5	6

$S - R$

n = 1
m = 3

Selection, product, and join for bags

- Selection, join: do not eliminate duplicate tuples in the result
- Product: the tuple rs will appear mn times
  - a tuple r appears m times in a relation R
  - a tuple s appears n times in a relation S

R

A	B	C
1	2	5
3	4	6
1	2	7
1	2	7

A	B	C
3	4	6
1	2	7
1	2	7

$\sigma_{C\geq 6}(R)$

R

A	B
1	2
1	2

S

B	C
2	3
4	5
4	5

A	B	C
1	2	3
1	2	3

$R \bowtie S$

A	R.B	S.B	C
1	2	2	3
1	2	2	3
1	2	4	5
1	2	4	5
1	2	4	5
1	2	4	5

$R \times S$

A	R.B	S.B	C
1	2	4	5
1	2	4	5
1	2	4	5
1	2	4	5

$R \bowtie_{R.B<S.B} S$

Extended operators of relational algebra

- Duplicate-elimination operator δ
  - turn a bag into a set by eliminating all but one copy of each tuple
- Aggregation operators
  - sum, average, ... (not operations of relational algebra)
  - but, used by the grouping operator (related with OLAP queries)
- Grouping operator γ
  - partition the tuples into groups according to their attribute value
- Sorting operator τ
  - turn into a list of tuples, sorted according to some attributes
- Outerjoin operator
  - variant of join that avoids losing dangling tuples

Duplicate elimination $\delta(R)$

A	B
1	2
3	4
1	2
1	2

R

A	B
1	2
3	4

$\boldsymbol{\delta(R)}$

Aggregation operators

- SUM: the sum of a column with numerical values
- AVG: the average of a column with numerical values
- MIN/MAX: the smallest or largest value
  - usually applied to a column with numerical values
  - for string values, the lexicographically first or last value
- COUNT: the number of values in a column

A	B
1	2
3	4
1	2
1	2

1. SUM(B) = 2 + 4 + 2 + 2 = 10
2. AVG(A) = (1 + 3 + 1 + 1) / 4 = 1.5
3. MIN(A) = 1
4. MAX(B) = 4
5. COUNT(A) = 4

Grouping operator $\gamma_L(R)$

- If there is grouping, the aggregation is within groups
- L: a list of elements, each of which is either
  - grouping attribute: attribute of R by which R will be grouped
  - aggregated attribute
    - attribute to which an aggregation operator is applied
    - an arrow and new name for the result of aggregation are appended
- Resulting relation of $\gamma_L(R)$ is constructed as follows:
  ① partition R into groups
    - if there are no grouping attributes, the entire relation R is one group

  ② for each group, produce one tuple consisting of:
    - grouping attributes' values
    - aggregations for the aggregated attributes, over all tuples of that group

Example

- Relation: StarsIn(title, year, starName)
- Query
  - "Find, for each star who has appeared in at least three movies, the earliest year in which they appeared"

aggregated attributes *grouping attribute*

title	year	starName
Star Wars	1977	Carrie Fisher
Empire Strikes Back	1980	Carrie Fisher
Return of the Jedi	1983	Carrie Fisher
Gone With the Wind	1939	Vivien Leigh
Wayne's World	1992	Dana Carvey
Wayne's World	1992	Mike Meyers

- Grouping expression
  - starName: grouping attribute
  - COUNT(title): aggregated attribute
    - selection $\sigma_{ctTitle>=3}(R')$ will be applied to the resulting relation R'
  - MIN(year): aggregated attribute

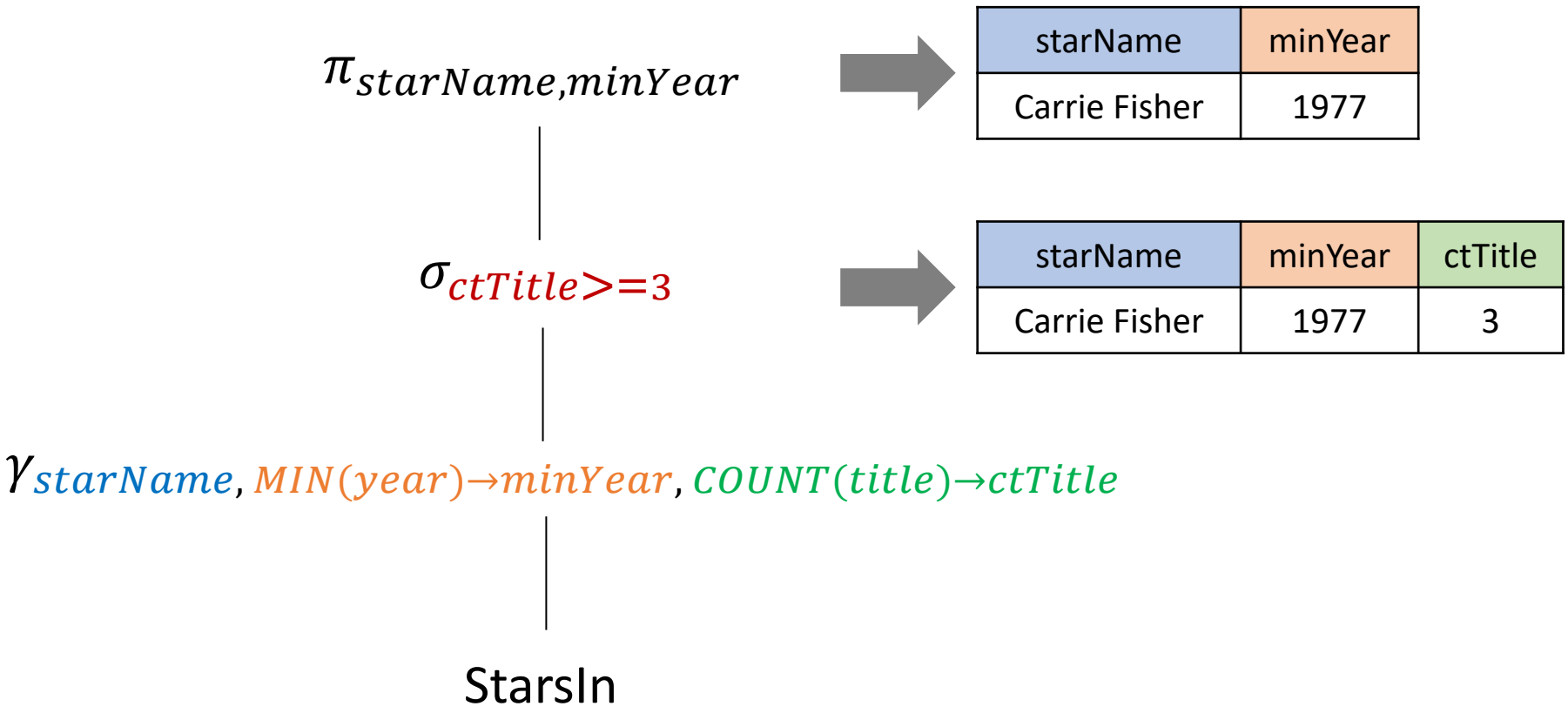
$$\gamma_{starName,\ MIN(year)\rightarrow minYear,\ COUNT(title)\rightarrow ctTitle}(R)$$

R

title	year	starName	
Star Wars	1977	Carrie Fisher	group1
Empire Strikes Back	1980	Carrie Fisher	
Return of the Jedi	1983	Carrie Fisher	
Gone With the Wind	1939	Vivien Leigh	group2
Wayne's World	1992	Dana Carvey	group3
Wayne's World	1992	Mike Meyers	group4

starName	minYear	ctTitle
Carrie Fisher	1977	3
Vivien Leigh	1939	1
Dana Carvey	1992	1
Mike Meyers	1992	1

Expression tree for the example query



Extending the projection operator $\pi_L(R)$

- Projection list L can have one of the following element
  - a single attribute of R
  - an expression $x \rightarrow y$ (renaming x to y)
  - an expression $E \rightarrow z$ (renaming the result of calculation E to z)
    - E: an expression involving attributes, constants, arithmetic operators, and string operators

A	B	C
0	1	2
0	1	2
3	4	5

R

A	X
0	3
0	3
3	9

$\pi_{A,B+C\rightarrow X}(R)$

X	Y
1	1
1	1
1	1

$\pi_{B-A\rightarrow X,C-B\rightarrow Y}(R)$

Sorting operator $\tau_L(R)$

- Result of $\tau_L(R)$: list of tuples (or sorted tuples)
  - R: a set or a bag
- It is often necessary to sort the tuples of a relation by L
  - L: one or more attributes of R (say $L = A_1, A_2, \ldots, A_n$)
  - e.g., SQL queries having ORDER BY
- The tuples of R are sorted first by their value of A_1
  - ties are broken according to the value of A_2
  - tuples that agree on both A_1 and A_2 are ordered according to their value of A_3 and so on
  - ties that remain after A_n is considered may be ordered arbitrarily

Outer joins

- Normal joins do not return dangling tuples as a result
  - dangling: fail to match any tuple of the other relation in the common attributes
- Outer joins add any dangling tuples in addition to $R \bowtie S$
  - useful in practice; so most of commercial DBMS's support it
  - added tuples must be padded with NULL in all the attributes that they do not possess, but appear in the join result
- Three kinds of outer joins
  - left outer join $⟕$: only dangling tuples of R are added to result
  - right outer join $⟖$: only dangling tuples of S are added to result
  - full outer join $⟗$: dangling tuples of R and S are added to result

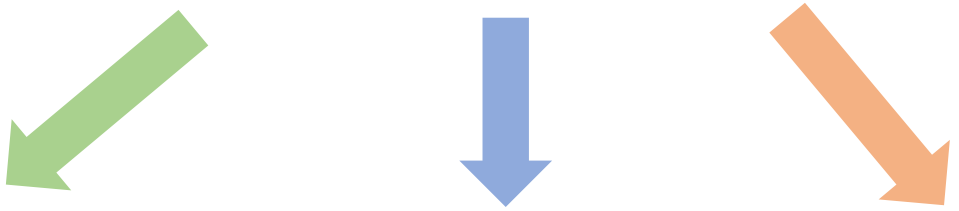
Example

U

A	B	C
1	2	3
4	5	6
7	8	9

B	C	D
2	3	10
2	3	11
6	7	12

V



left outer join

A	B	C	D
1	2	3	10
1	2	3	11
4	5	6	⊥
7	8	9	⊥

$U \ ⟕ \ V$

full outer join

A	B	C	D
1	2	3	10
1	2	3	11
4	5	6	⊥
7	8	9	⊥
⊥	6	7	12

$U \ ⟗ \ V$

right outer join

A	B	C	D
1	2	3	10
1	2	3	11
⊥	6	7	12

$U \ ⟖ \ V$

⊥ indicates a NULL value

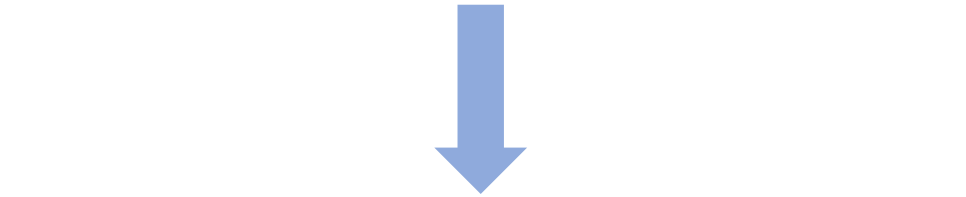
Example

U

A	B	C
1	2	3
4	5	6
7	8	9

B	C	D
2	3	10
2	3	11
6	7	12

V



full outer theta join

A	U.B	U.C	V.B	V.C	D
4	5	6	2	3	10
4	5	6	2	3	11
7	8	9	2	3	10
7	8	9	2	3	11
1	2	3	$\perp$	$\perp$	$\perp$
$\perp$	$\perp$	$\perp$	6	7	12

$$U \; \exists\!\!\!\bowtie\!\!\!\sqsubset_{A>V.C} \; V$$

Datalog

Datalog: logical query language for relations

- Datalog ("Database logic") consists of if-then rules
  - each **predicate** takes a fixed number of arguments
  - atom: a predicate followed by its arguments
    - syntax of atom is just like that of function calls in conventional PL
    - e.g., $P(x_1, x_2, \ldots, x_n)$
    - a predicate returns a boolean value
- Relation R with n attributes is represented as a predicate
  - $R(a_1, a_2, \ldots, a_n)$ returns TRUE, if $(a_1, a_2, \ldots, a_n)$ is a tuple of R
  - it returns FALSE otherwise
  - e.g., R(1,2) and R(3,4) are TRUE;

    any other combination of values x and y, R(x,y) is FALSE

A	B
1	2
3	4

Example

- R(x,y) tells whether the tuple (x,y) is in R for any x and y
- For the particular instance of R
  - R(x,y) returns TRUE when either
    - x = 1 and y = 2, or
    - x = 3 and y = 4
  - it returns FALSE otherwise

A	B
1	2
3	4

R

  - R(1,z) returns TRUE, if z = 2; returns FALSE otherwise

- Note: the instance is changed, the atom works differently

Arithmetic atom

- Relational atoms: return T or F according to an instance
  - e.g., atoms in p.21-22
- Arithmetic atoms: comparison between two arithmetic expressions
  - e.g., $x < y$
    - return TRUE, if x=1, and y=2
    - return FALSE, if x=2, and y=1
  - e.g., $x + 1 \geq y + 4 \times z$
- Both atoms take arguments and return a boolean value

Datalog rules and queries

- Datalog rules consist of
  ① a relational atom called the head, followed by
  ② the symbol ←, which we often read "if," followed by
  ③ a body consisting of one or more atoms, called subgoals
    - each is either relational or arithmetic atom
    - subgoals are connected by AND
    - any subgoal may optionally be preceded by the logical operator NOT
- e.g.,
  – define the set of long movies of at least 100 minutes long
    - 1st subgoal: let (t,y,l,g,s,p) be a tuple in the current instance of Movies
    - 2nd subgoal: true whenever the length of a Movies tuple is at least 100

$$LongMovie(t, y) \leftarrow Movies(t, y, l, g, s, p) \text{ AND } l \geq 100$$

$LongMovie :=$ $\pi_{title,year}(\sigma_{length \geq 100}(Movies))$ in relational algebra

Query in Datalog

- Collection of one or more rules

- If there is only one relation in the rule heads
  - the value of that relation is taken to be the answer to the query
  - e.g., LongMovie

- If there is more than one relation among the rule heads
  - one of these relations is the answer to the query
  - the other relations assist in the definition of the answer

- The query result is usually named as $Answer$

Safety of Datalog rules

- Query result MUST be a finite relation
  - e.g., $Answer(t, y) \leftarrow \text{NOT } Movies(t, y, l, g, s, p)$ (not finite)
- Safety condition
  - every variable that appears anywhere in the rule must appear in some nonnegated, relational subgoal of the body
  - c.f., negated subgoals: those with NOT in front of them
- e.g., $LongMovie(t, y) \leftarrow Movies(t, y, l, \_, \_, \_)$ AND $l \geq 100$
  - 1st subgoal: nonnegated, relational subgoal
    - the variables represented by underscores are anonymous variables
  - t and y in the head appears in the 1st subgoal of the body
  - l appears in an arithmetic subgoal, but also appears in 1st subgoal
  - thus, the rule is safe

Example

- $P(x, y) \leftarrow Q(x, z)$ AND NOT $R(w, x, z)$ AND $x < y$

- y does not appear in any nonnegated, relational subgoal
  - its appearance in the arithmetic subgoal dose not help to limit the possible values of y to a finite set

- w appears only in a negated, relational subgoal

- Thus, the rule is not safe and cannot be used in Datalog

Consistency in Datalog rules

- Assignment of tuples for each nonnegated, relational subgoal is consistent
  - when it assigns the same value to each occurrence of any one variable

- e.g., $P(x, y) \leftarrow Q(x, z)$ AND $R(z, y)$ AND NOT $Q(x, y)$
  - relation Q contains two tuples (1,2) and (1,3)
  - relation R contains two tuples (2,3) and (3,1)

	Tuple for Q(x,z)	Tuple for R(z,y)	Consistent Assignment?	NOT Q(x,y) True?	Resulting Head
1)	(1,2)	(2,3)	Yes	No; (x,y)=(1,3)	–
2)	(1,2)	(3,1)	No; z=2,3	Irrelevant	–
3)	(1,3)	(2,3)	No; z=3,2	Irrelevant	–
4)	(1,3)	(3,1)	Yes	Yes; (x,y)=(1,1)	P(1,1)

Example of multiple rules and bag-valued relations

- $H(x, y) \leftarrow S(x, y)$ AND $x > 1$
- $H(x, y) \leftarrow S(x, y)$ AND $y < 5$

- 1st rule: puts each of three tuples into H
- 2nd rule: puts only the tuple (2,3) into H
  - (4,5) does not satisfy the condition $y < 5$

B	C
2	3
4	5
4	5

S

x	y
2	3
2	3
4	5
4	5

H

Relational algebra and Datalog

- Any **single** safe Datalog rule can be expressed in relational algebra
  - skip the proof here

- Datalog queries are more powerful than relational algebra
  - when multiple rules are allowed to interact
  - can express recursions that are not expressable in the algebra

Boolean operations

- Assumption: R and S are relations with n attributes

- $R \cup S$
  - $Ans(a_1, a_2, \ldots, a_n) \leftarrow R(a_1, a_2, \ldots, a_n)$
  - $Ans(a_1, a_2, \ldots, a_n) \leftarrow S(a_1, a_2, \ldots, a_n)$

- $R \cap S$
  - $Ans(a_1, a_2, \ldots, a_n) \leftarrow R(a_1, a_2, \ldots, a_n)$ AND $S(a_1, a_2, \ldots, a_n)$

- $R - S$
  - $Ans(a_1, a_2, \ldots, a_n) \leftarrow R(a_1, a_2, \ldots, a_n)$ AND NOT $S(a_1, a_2, \ldots, a_n)$

Projection

- We want to project Motives onto its first three attributes

$$Movies(title, year, length, genre, studioName, producerC\#)$$

$$P(t, y, l) \leftarrow Movies(t, y, l, g, s, p)$$

Selection

- $\sigma_{length \geq 100 \text{ AND } studioName="Fox"}(Movies)$

$$Ans(t,y,l,g,s,p) \leftarrow Movies(t,y,l,g,s,p) \text{ AND } l \geq 100 \text{ AND } s = "Fox"$$

- $\sigma_{length \geq 100 \text{ OR } studioName="Fox"}(Movies)$

$$Ans(t,y,l,g,s,p) \leftarrow Movies(t,y,l,g,s,p) \text{ AND } l \geq 100$$
$$Ans(t,y,l,g,s,p) \leftarrow Movies(t,y,l,g,s,p) \text{ AND } s = "Fox"$$

- $\sigma_{\text{NOT}(length \geq 100 \text{ OR } studioName = "Fox")}(Movies)$
- $\sigma_{(\text{NOT}(length \geq 100) \text{ AND } (\text{NOT}(studioName = "Fox"))}(Movies)$ by DeMorgan's laws
- $\sigma_{length < 100 \text{ AND } studioName \neq "Fox"}(Movies)$

$$Ans(t,y,l,g,s,p) \leftarrow Movies(t,y,l,g,s,p) \text{ AND } l < 100 \text{ AND } s \neq 'Fox'$$

- $\sigma_{\text{NOT}(length \geq 100 \text{ AND } studioName = "Fox")}(Movies)$
- $\sigma_{(\text{NOT}(length \geq 100) \text{ OR } (\text{NOT}(studioName = "Fox"))}(Movies)$
- $\sigma_{length < 100 \text{ OR } studioName \neq "Fox"}(Movies)$

$$Ans(t,y,l,g,s,p) \leftarrow Movies(t,y,l,g,s,p) \text{ AND } l < 100$$
$$Ans(t,y,l,g,s,p) \leftarrow Movies(t,y,l,g,s,p) \text{ AND } s \neq 'Fox'$$

Product

- Product of two relations $R \times S$ can be expressed by a single Datalog rule

- e.g., $P := R \times S$ for $R(a, b, c)$ and $S(x, y, z)$

$$P(a, b, c, x, y, z) \leftarrow R(a, b, c) \text{ AND } S(x, y, z)$$

Joins

- Natural join between R(A,B) and S(B,C,D)

$$J(a,b,c,d) \leftarrow R(a,b) \text{ AND } S(b,c,d)$$

- Theta join between U(A,B,C) and V(B,C,D)

$$U \bowtie_{A<D \text{ AND } U.B \neq V.B} V$$

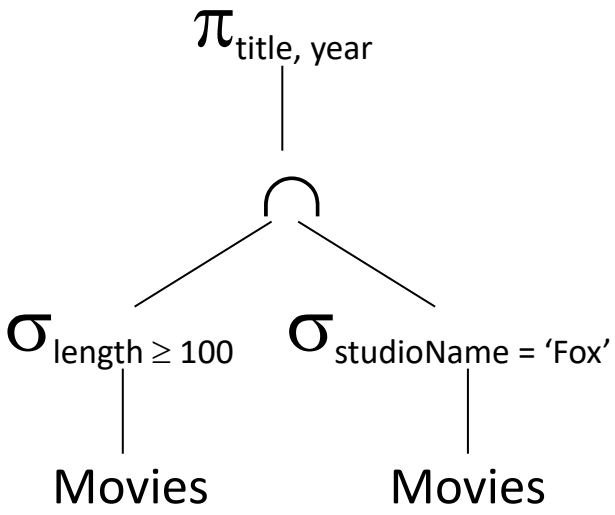
$$J(a,ub,uc,vb,vc,d) \leftarrow U(a,ub,uc) \text{ AND } V(vb,vc,d) \text{ AND } a < d \text{ AND } ub \neq vb$$

$$U \bowtie_{A<D \text{ OR } U.B \neq V.B} V$$

$$J(a,ub,uc,vb,vc,d) \leftarrow U(a,ub,uc) \text{ AND } V(vb,vc,d) \text{ AND } a < d$$

$$J(a,ub,uc,vb,vc,d) \leftarrow U(a,ub,uc) \text{ AND } V(vb,vc,d) \text{ AND } ub \neq vb$$

Multiple operations with Datalog



- $\pi_{\text{title, year}}(\sigma_{\text{length} \geq 100}(\text{Movies}) \cap \sigma_{\text{studioName = 'Fox'}}(\text{Movies}))$

- $W(t, y, l, g, s, p) \leftarrow Movies(t, y, l, g, s, p)$ AND $l \geq 100$
- $X(t, y, l, g, s, p) \leftarrow Movies(t, y, l, g, s, p)$ AND $s = 'Fox'$
- $Y(t, y, l, g, s, p) \leftarrow W(t, y, l, g, s, p)$ AND $X(t, y, l, g, s, p)$
- $Ans(t, y) \leftarrow Y(t, y, l, g, s, p)$

- $Ans(t, y) \leftarrow Movies(t, y, l, g, s, p)$ AND $l \geq 100$ AND $s = 'Fox'$

Comparison between Datalog and relational algebra

- Datalog rules can express recursions
  - c.f., relational algebra cannot do that

- e.g., for a relation $Edge(X, Y)$,
  - Edge(X,Y): a directed edge from node X to node Y
  - Path(X,Y): a path of length one or more from node X to node Y
  - Datalog can express its transitive closure, i.e., $Path(X, Y)$

1. $Path(X, Y) \leftarrow Edge(X, Y)$
2. $Path(X, Y) \leftarrow Edge(X, Z)$ AND $Path(Z, Y)$

  - rule (1): every edge is a path
  - rule (2): if there is an edge from X to some Z and a path from Z to Y, then there is also a path from X to Y
  - repeating application of rule (2) can discover all possible path facts

Thank you

Q & A